

实验七 Verilog 实验：组合逻辑

姓名：庄环嘉 学号：22010022088

(1) LAB 1: 用 Verilog 实现一个简单 ALU

一、实验内容

输入：两个 4 位二进制数，代表两个操作数 A, B；一个 3 位控制信号 operation，代表 ALU 要进行的运算。本实验中，ALU 可以实现 8 种运算：

输出：4 位结果，1 位进位

二、代码实现及仿真图像

1. 源代码

```
`timescale 1ns / 1ps
module ALU(
    input [3:0] A,
    input [3:0] B,
    input [2:0] operation,
    output reg [3:0] result,
    output reg cout
);

    always @(*) begin
    case(operation)
        //A+B 3'b000: begin result <= A + B;
            end
        //A-B 3'b001: begin result <= A - B;
            end
        //B+1 3'b010: begin result <= B + 1;
            end
        //B-1 3'b011: begin result <= B - 1;
            end
        //NOT A 3'b100: begin result <= ~A;
            end
        //A XOR B 3'b101: begin result <= A ^ B;
            end
    end
```

```

//A AND B 3'b110: begin result <= A & B;
    end
//A OR B 3'b111: begin result <= A | B;
    end
endcase //完成八种运算
end
endmodule

```

2. 仿真代码

```

`timescale 1ns / 1ps
module ALU_tb;
    reg [3:0] A;
    reg [3:0] B;
    reg [2:0] operation;
    wire [3:0] result;
    wire cout;

    ALU ALU (
        A,
        B,
        operation,
        result,
        cout
    );

    initial begin
        A <= 4'b0001; B <= 4'b0010;
        operation <= 000; // A + B 0011 没有进位

        #10 A <= 4'b1100; B <= 4'b1000;
        operation <= 000; // A + B 0100 产生进位

        #10 A <= 4'b0001; B <= 4'b0001;
        operation <= 001; // A - B 0000

        #10 A <= 4'b0000; B <= 4'b0001;
        operation <= 001; // A - B < 0 1111 产生借位

        #10 A <= 4'b0000; B <= 4'b0001;
        operation <= 010; // B + 1 0010

        #10 A <= 4'b0000; B <= 4'b1111;

```

```

operation <= 010; // B + 1 0000 产生进位

#10 A <= 4'b0000; B <= 4'b0001;
operation <= 011; // B - 1 0000

#10 A <= 4'b0000; B <= 4'b0000;
operation <= 011; // B - 1 1111 产生借位

#10 A <= 4'b1001; B <= 4'b0001;
operation <= 100; //NOT A 0110

#10 A <= 4'b0001; B <= 4'b0010;
operation <= 101; //A XOR B

#10 A <= 4'b0001; B <= 4'b0001;
operation <= 110; //A AND B

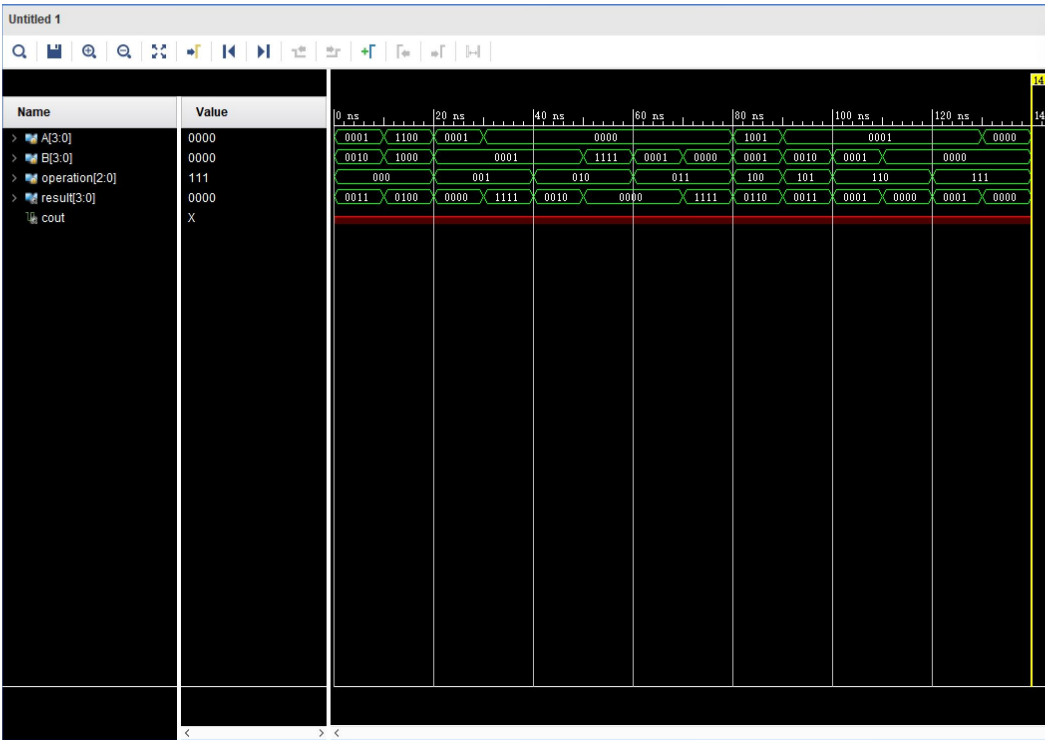
#10 A <= 4'b0001; B <= 4'b0000;
operation <= 110; //A AND B

#10 A <= 4'b0001; B <= 4'b0000;
operation <= 111; //A OR B

#10 A <= 4'b0000; B <= 4'b0000;
operation <= 111; //A OR B
#10 $stop;
end
endmodule

```

3. 仿真图像



三、实验结果

在这个实验中，ALU 接受两个 8 位操作数（operandA 和 operandB）、一个 3 位操作码（opcode），并产生一个 8 位结果（result）和一个零标志（zero）。它支持加法、减法、位与、位或、位异或、左移和右移操作。如果结果为零，零标志将被设置为 1。在本实验中我学习到如何用 Verilog 进行功能验证和生成仿真波形，使逻辑思路更加具象化。

(2) 自编实验——半加法器

一、实验内容

设计一位半加法器，即两个一位的二进制数相加，得到其正常相加的结果的最后一位。

二、代码实现及仿真图像

1. 源代码

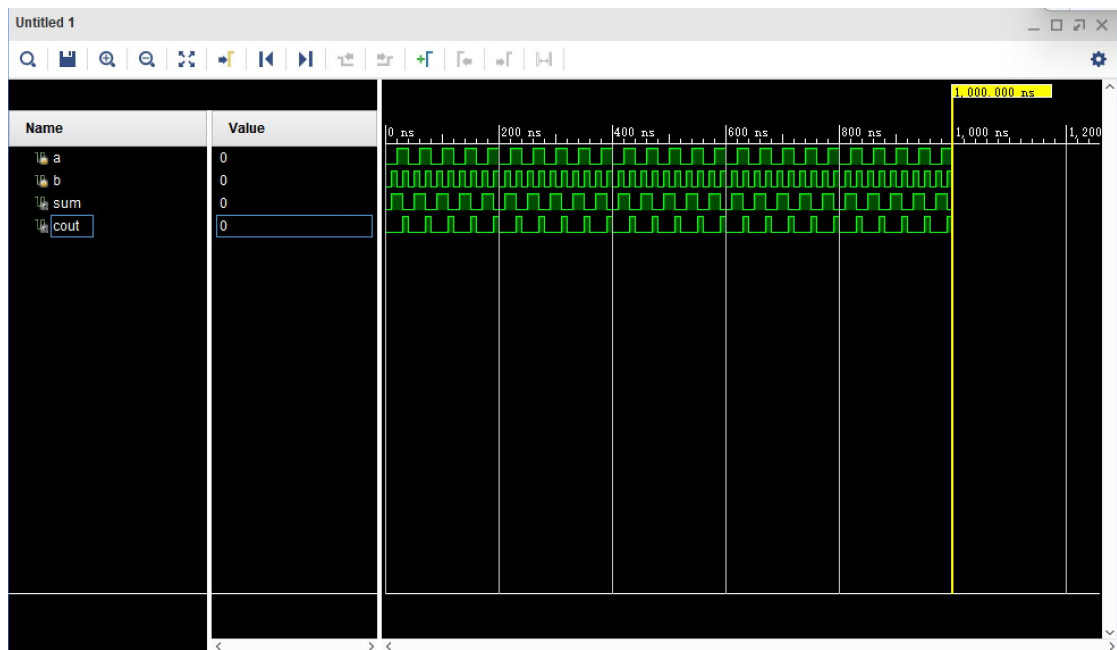
```
`timescale 1ns / 1ps
module bjqxb(a,b,sum,cout);
    input a,b;
    output sum,cout;
    reg sum,cout;
    always @(a or b)
        begin
            case({a,b})
                2'b00:begin
                    sum=0;cout=0;
                end
                2'b01:begin
                    sum=1;cout=0;
                end
                2'b10:begin
                    sum=1;cout=0;
                end
                2'b11:begin
                    sum=0;cout=1;
                end
            endcase
        end
endmodule
```

2. 仿真代码

```
`timescale 1ns / 1ps
module bjqxwsimu;
    reg a,b;
    wire sum,cout;
```

```
bjqxw sl(a,b,sum,cout);  
initial  
begin  
    a=0;b=0;  
end  
always #10 {a,b}={a,b}+1;  
endmodule
```

3. 仿真图像



三、实验结果

这个半加法器是一个基本的数字电路,用于将两个一位的二进制数相加。它有两个输入:两个待相加的二进制位 (A 和 B) , 并有两个输出: 和 (Sum) 和进位 (cout) 。这个半加法器使用异或门 (XOR) 来计算和 (Sum) , 使用与门 (AND) 来计算进位 (cout) 。

以此数字电路为基础, 可以构建更大的数字电路, 如加法器和多位加法器。